Project Assignment 2: Classification

Logistic Regression from Scores to Decisions

Format: Portfolio project in your own Git repository
Estimated effort: 4-6 hours for modeling, plus documentation and peer validation
Prerequisite: Project 1's two-feature linear regression and gradient descent

Purpose

In Project 1, you predicted a continuous outcome using a weighted sum of features. Now you will turn a weighted sum into a probability, learn its coefficients using binary cross-entropy and gradient descent, and turn that probability into a class prediction.

You will implement logistic regression with NumPy, compare it with scikit-learn, and explain how a classification threshold changes the errors a model makes. You do not need to implement the Normal Equation: it is not the fitting method for this logistic regression assignment.

As in Project 1: Linear Regression, the deliverable is a reproducible, professionally presented investigation, not just a Python script. Treat your repository as part of your growing personal portfolio. An employer should be able to understand your question, evidence, decisions, challenges, findings, and limitations, and another person should be able to reproduce your work.

Dataset and boundaries

Use scikit-learn's bundled **Wisconsin Diagnostic Breast Cancer** dataset, loaded with `sklearn.datasets.load_breast_cancer()`. No account, external CSV, or dataset download is required after installing scikit-learn. Read its `DESCR` and include a brief source acknowledgement in your report. The measurements describe cell nuclei from breast-mass samples, not ordinary patient demographics.

Use exactly these two input features, in this order:

1. `mean radius`
2. `mean texture`

The loader's original labels are **0 = malignant, 1 = benign**. Recode the target for this assignment:

```
y = (data.target == 0).astype(int)
```

Throughout your work, **1 = malignant (positive)** and **0 = benign (negative)**. State this explicitly in your outputs. Do not use any other features, the target, or a row identifier as a predictor.

Educational use only: this small, historical dataset and two-feature model are not a diagnostic tool or evidence of clinical readiness. Do not use personal medical information or make patient-specific recommendations.

Setup

Use Python 3.10 or newer in the same environment you use to run your work. From the repository root in PowerShell:

```
cd .\Project_02_Classification\logistic-regression
python -m pip install -r ..\assignment_requirements.txt
```

NumPy is required for your own training implementation. You may use scikit-learn for loading, splitting, scaling, the reference model, the baseline, and evaluation metrics. You may use Matplotlib for the loss plot.

Part 1: Connect the equation to a prediction

Use the interactive explorer to investigate the following four settings. Its one-feature examples are separate from the two-feature dataset you will train on.

Case	Intercept beta_0	Coefficient beta_1	Input x	Threshold
A	-1.0	0.8	2.0	0.50
B	-1.0	0.8	2.0	0.70
C	-1.0	-0.8	2.0	0.50
D	1.0	0.0	2.0	0.50

For each case, record the score `z = beta_0 + beta_1*x`, probability `p = 1 / (1 + exp(-z))`, and predicted class using `p >= threshold`. Show your calculation for case A and report probabilities to four decimals. Explain what changes between A and B, what reversing the coefficient does, and why D has a constant probability for every x. The GUI changes coefficients manually; it does not train them.

If your computer cannot open a desktop GUI, calculate the same four cases directly in Python and explain the corresponding curve behavior. A screenshot is not required and does not replace the calculations.

Part 2: Prepare a leakage-free experiment

1. Load the dataset, select the two named columns, and recode the target.
2. Report feature names, missing-value counts, and overall class counts. No imputation is needed for this dataset if you have loaded it correctly.
3. Split **row indices** with `train_test_split` in the exact sequence below:
   - First split all indices into development and test indices with `test_size=0.20`, `stratify=y`, and `random_state=42`.
   - Split the returned development indices into training and validation indices with `test_size=0.25`, `stratify=y[development_indices]`, and `random_state=42`.
   - Keep the returned order; do not sort or shuffle the indices again.
4. Print the number of rows and each class count in each partition. Verify that the three index sets do not overlap and together cover every row.
5. Fit `StandardScaler` on the **training features only**. Use that fitted scaler to transform training, validation, and test features. Both logistic models must receive the same standardized data.

Do not inspect test performance to choose features, learning settings, or a threshold. Keep the test partition untouched until Part 5.

Part 3: Implement NumPy logistic regression

Implement a two-feature binary logistic model with an intercept:

```
z = beta_0 + beta_1*x1_scaled + beta_2*x2_scaled
p = sigmoid(z)
```

Requirements:

- Use full-batch gradient descent and mean binary cross-entropy (log loss).

- Initialize the intercept and both coefficients to zero. Start with `learning_rate=0.1` and `epochs=20000`. Use no regularization or class weights.
- If `X_aug` is the standardized matrix with a leading column of ones and `beta` contains all three coefficients, the gradient is `X_aug.T @ (p - y_train) / n_train`.
- Update all coefficients from that same gradient each epoch.
- Use stable probability and loss calculations. For example, `exp(-logaddexp(0, -z))` evaluates sigmoid safely and `mean(logaddexp(0, z) - y_train*z)` evaluates the training loss without taking the logarithm of zero.
- Record the loss before training and after each update, including the final coefficients' loss. Print the initial and final training loss and save a labeled loss-versus-epoch plot.
- Print all three learned coefficients, identifying them as **standardized-unit** coefficients. Conversion to original feature units is not required.

Write the gradient-descent loop yourself. Do not use a library estimator, automatic differentiation, or another optimizer to fit your NumPy model. You may reuse your Project 1 code patterns with appropriate acknowledgement. If you change the suggested learning rate or epoch count to resolve convergence, document the change and show convergence evidence using training/validation data.

In your report, explain why minimizing cross-entropy learns the coefficients and how it relates to maximizing the observed labels' likelihood. A formal derivation of maximum likelihood is not required.

Part 4: Compare training implementations and a baseline

Train these additional models on the **same training rows**:

- **scikit-learn reference:** use `LogisticRegression(C=np.inf, solver="lbfgs", max_iter=10000, tol=1e-9)`. `C=np.inf` removes the regularization effect for this comparison. Do not use the default finite-C model, class weights, or a second scaler. Let the estimator fit its own intercept; do not pass it your column of ones.
- **Majority baseline:** use `DummyClassifier(strategy="prior")`. It learns training class proportions; at threshold 0.50 it predicts the majority class. Use its probabilities when computing log loss, not hard class labels.

For both logistic models, compare the three coefficients and calculate the mean absolute difference between their **validation probabilities**. They should be close because the models minimize the same unregularized objective. Investigate large differences rather than copying the reference coefficients into your NumPy model.

Use the probability of **class 1** from `predict_proba`; verify the relevant column against `classes_`. Explain why the baseline can have reasonable accuracy while failing to identify malignant samples.

Part 5: Select a threshold, then evaluate

1. On the **NumPy model's validation probabilities only**, evaluate exactly these thresholds: `0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80`.
2. Select the threshold with the highest validation F1 score. If multiple candidates tie for the maximum, select the **smallest** of those thresholds.
3. Freeze both trained models, the scaler, and that threshold. Do not refit on training plus validation data for this assignment.
4. Apply that same selected threshold to both logistic models. It is selected from the NumPy model, not separately optimized for the scikit-learn model.
5. Evaluate the following five configurations on validation data and then on the test data in one final evaluation stage:
   - Baseline at 0.50 (`policy=default`).
   - NumPy at 0.50 (`policy=default`).

- NumPy at the selected threshold (`policy=selected`).
- Scikit-learn at 0.50 (`policy=default`).
- Scikit-learn at the selected threshold (`policy=selected`).

For every configuration report TN, FP, FN, TP, accuracy, precision, recall, F1, and log loss. Use `confusion_matrix(..., labels=[0, 1])`: rows are actual classes and columns are predicted classes. Label matrices with the class meanings. Compute log loss from probabilities, never thresholded predictions. Use `zero_division=0` for precision, recall, and F1 when a denominator is zero, and explain this convention for the baseline.

The selected threshold may or may not improve test F1 or recall. Report what happens honestly; do not change it after seeing the test results. Comparing the two predeclared policies on the test set is allowed; using that comparison to select a new policy is not. F1 is the exercise's selection rule, not a clinical cost function or a claim that false positives and false negatives are equally consequential.

Part 6: Export results and explain them

Print a readable comparison table and export these files. Report accuracy, precision, recall, and F1 as decimals in `[0, 1]`, not percentages. Log loss is nonnegative and is not restricted to `[0, 1]`. Retain at least six decimal places for metrics in CSVs; confusion counts must be integers.

Required outputs

File	Required content
`reports/threshold_results.csv`	Seven validation rows, one per candidate threshold, with columns `threshold,tn,fp,fn,tp,accuracy,precision,recall,f1`
`reports/assignment_results.csv`	Ten rows: the five configurations for each of validation and test, with columns `split,model,policy,threshold,tn,fp,fn,tp,accuracy,precision,recall,f1,log_loss`
`reports/training_loss.png`	NumPy training loss versus epoch, with title and labeled axes

Use `split` values `validation` and `test`; use `model` values `baseline`, `numpy`, and `sklearn`. Keep `default` and `selected` as separate rows even if the selected threshold equals 0.50. Create the output folder if necessary. Make data loading and output paths work regardless of the terminal's current directory, for example by resolving paths relative to your script.

Written report

Submit approximately 500-750 words, plus the four-case calculation table and a coefficient comparison table. Address all of these:

1. The Part 1 calculations and explanations: score, sigmoid, threshold, and manually set versus learned coefficients.
2. Dataset source, positive-class coding, class balance, and how your split and scaling prevent leakage.
3. Training loss behavior, the cross-entropy/likelihood connection, both sets of coefficients, and the mean validation-probability difference. Interpret one coefficient as a change in **log-odds per training-standard-deviation** increase, holding the other feature fixed, not a constant probability change or causal effect.
4. Baseline versus logistic results. Explain why accuracy alone is insufficient and interpret one labeled confusion matrix in plain language.
5. The selected threshold, why validation chose it, and how its test FP/FN, precision, recall, and F1 compare with the default policy. Explain why changing the threshold does not change probabilities or log loss.

6. Why this is not ready for clinical use: discuss false-negative consequences and at least two limitations, such as limited features, sample size, representativeness, unverified calibration, or missing external validation.

Add a short research-process section, outside the suggested word count, describing challenges you actually encountered, how you investigated them, what you changed, and what evidence confirmed the resolution. If there were no major errors, describe a design decision and how you checked it; do not invent problems. Acknowledge sources and reused code.

Professional portfolio repository

Publish your work in your own Git repository. Use a meaningful project name, such as **Breast Mass Classification: Logistic Regression and Decision Thresholds**, rather than "Assignment: Module 2" or a generic assignment label. The course identifies this as Project Assignment 2; your public-facing title should describe the work, not the submission number.

Your repository must include:

- A root `README.md` that serves as both a professional project overview and the complete reproduction guide described below.
- Your end-to-end Python script. You may retain `assignment_logistic_regression.py` or choose a descriptive name such as `logistic_regression_analysis.py`; document the exact filename and run command.
- The three required files in `reports/`, with readable labels and explanations.
- `memo.md` or `memo.pdf` containing the report, calculation and coefficient tables, and research-process section. Link to it from the README.
- A dependency file such as `requirements.txt`, sufficient to install the libraries you actually use. Record tested Python and package versions.
- An appropriate `.gitignore` excluding environments, caches, secrets, and the completed peer-validation form. Keep the form outside the repository.

Organize files logically, use descriptive names, and remove abandoned drafts and unrelated course files. Use meaningful Git commits showing your progress. Do not upload your virtual environment, credentials, package caches, or copies of the bundled dataset. Keep your NumPy implementation readable and separate from the reference fit so the training logic can be checked.

README: a beginner must be able to reproduce the work

Assume your reader has never used a terminal, Python, or Git, and you do not know whether they use Windows or an Apple Mac. Do not assume any installed software or unstated technical knowledge. Your README must provide:

1. A plain-language question, dataset source and context, positive-class definition, comparison methods, and a concise summary of actual findings and limitations. Explain the educational-only, nonclinical scope.
2. Prerequisites and links to obtain Python and Git; how to verify installation; and where/how to open a terminal on Windows and macOS.
3. Your actual repository URL and step-by-step cloning instructions, followed by how to navigate into the correct project folder. Explain any placeholders.
4. Clearly separated **Windows PowerShell** and **macOS Terminal** instructions for environment creation, activation, dependency installation, and execution. Use OS-appropriate commands and path separators. Explain what to do if PowerShell activation is blocked, without requiring a permanent security policy change. Do not rely on VS Code being installed.
5. The exact commands and their working directories, the tested Python and library versions, and any nondefault training settings. Explain that the dataset loads from scikit-learn and requires no separate download.

6. What successful execution looks like: approximate runtime on your machine, expected printed tables, names and locations of the two CSVs and loss plot, and how to open those outputs. Explain acceptable numerical variation.
7. Troubleshooting for likely problems, including an unrecognized Python command, missing dependencies, and running from the wrong folder.

The README must be self-contained; a separate `run_instructions.txt` is not required and cannot replace it. A reader must not need your spoken explanation, a private message, an absolute path from your computer, or access to your instructor's local files.

Independent peer validation

Use the course's Project Replication & Peer Verification Form. Before submission:

1. Send a classmate the repository link and the branch/version to validate. They must obtain a fresh copy on their own computer and follow only the README, including installation, setup, execution, and checking outputs.
2. **Do not help them.** They cannot ask you questions about the steps. Do not provide live guidance, remote control, private setup notes, or dependencies already configured on your computer as a substitute for documentation.
3. The reviewer records their operating system, Python version, tested branch and commit identifier, outcomes, and any blockers. If the form has no space, attach a short verification notes page to the same PDF.
4. If a step fails or is ambiguous, the reviewer records the issue. Revise the repository and README, then have the peer repeat validation independently on the updated version before signing. Record the final successful commit.
5. Both parties complete the form honestly. Validate the peer's actual operating system; do not claim both platforms were tested unless they were. Written instructions for both Windows and macOS are required even if only one was available for the independent run.

Follow the replication policy stated on the course form. Keep the completed form and signatures private: **submit them to Canvas as a PDF, never to Git**. Your public README may mention that replication was completed and the tested platform, but must not include the signed form or private reviewer details.

Submit on Canvas

- **Your Git repository link**, with the branch and final validated commit identifier. Ensure the instructor can access that version.
- **The completed peer-validation form as a PDF**, including any verification notes. Upload it directly to Canvas, not to the repository.
- A short submission note identifying any optional extensions and where to find them, or stating that you completed the required scope only.

A ZIP or script upload is not a substitute for the Git link and validation PDF.

Optional portfolio extensions

You may go beyond the required work: compare another appropriate classifier, study regularization or additional features, or add a clearly explained visualization. Keep the prescribed two-feature experiment and its required outputs intact and easy to find. Label extra analyses separately, cite sources, explain why you chose them, and compare them fairly without tuning on the held-out test results. Extensions that require tuning need a separate development/evaluation plan established before examining their test results. You are not required to try every classification algorithm.

Tell the instructor about extensions in your Canvas submission note so they know what to expect. Extra complexity is not a substitute for reproducibility, correct interpretation, or completing the required project.

See the student rubric for the expectations used to review completeness, correctness, evidence, and interpretation. A particular accuracy or a better test result at the selected threshold is not a requirement.